

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐẠI HỌC QUỐC GIA TP.HCM
KHOA ĐIỆN TỬ VIỄN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

TRUYỀN THÔNG SỐ

MÃ HÓA LDPC

GIÁO VIÊN	NHÓM SINH VIÊN
TS. Đặng Lê Khoa TS. Nguyễn Minh Trí ThS. Ngô Thanh Hải	Trần Văn Minh Phương – 22207075 Giao Hữu Lực – 23207080 Lê Quang Thông – 23207113 Đặng Bá Trần Trung – 23207121

TP. Hồ Chí Minh - 04/2026

MỤC LỤC

I. LÝ THUYẾT	2
1. Linear Block Code	2
2. Mã LDPC	3
3. Giải Đồ Tanner	4
4. Thuật Toán Bit-Flipping	4
II. THỰC HÀNH	6
Câu 1	6
Câu 2	6
Câu 3	8

DANH MỤC HÌNH ẢNH

Hình 1: Quá trình mã hóa tuyến tính	2
Hình 2: Ma trận kiểm tra chẵn lẻ H	3
Hình 3: Giải đồ Tanner	4

I. LÝ THUYẾT

1. Linear Block Code

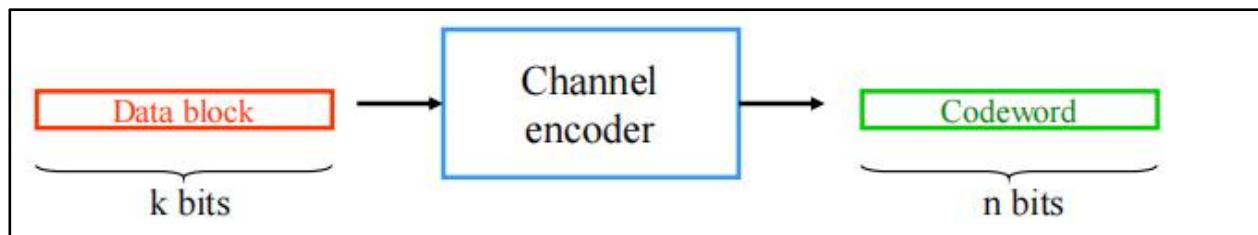
Linear Block Code là một lớp mã sửa lỗi tuyến tính, trong đó các từ mã được tạo ra từ tổ hợp tuyến tính của các bit thông tin, nhằm mục đích phát hiện và sửa lỗi trong quá trình truyền dữ liệu.

Luồng bit thông tin được chia thành các khối k bit.

Mỗi khối được mã hóa thành một khối lớn hơn gồm n bit.

Các bit đã mã hóa được điều chế và gửi qua kênh.

Quy trình ngược lại được thực hiện ở phía thu.



Hình 1: Quá trình mã hóa tuyến tính

Cụ thể từ mã được xác định như sau:

$$c = u * G$$

Trong đó:

- + c : vector từ mã có độ dài n
- + u : vector thông tin ban đầu có độ dài k
- + G : ma trận sinh kích thước $k * n$

Một đặc điểm quan trọng của Linear Block Code là mọi codeword đều thỏa mãn:

$$c * H^T = 0$$

với H là ma trận kiểm tra chẵn lẻ (Parity-Check Matrix).

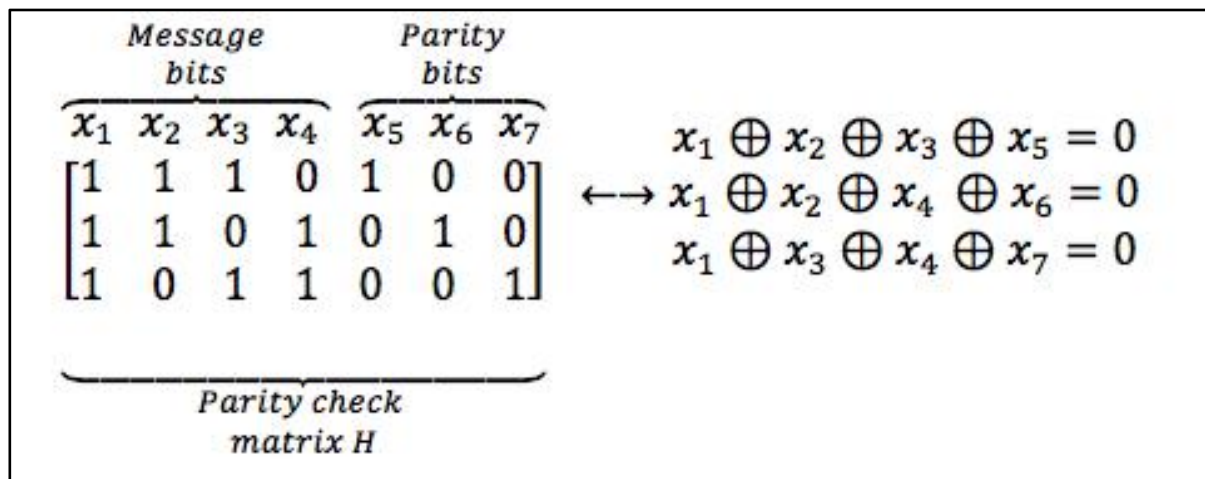
Do tính chất tuyến tính, tổng của hai từ mã bất kỳ vẫn là một từ mã hợp lệ, giúp đơn giản hóa việc phân tích cũng như quá trình phát hiện và sửa lỗi.

Việc sử dụng trường nhị phân Galois Field $GF(2)$ giúp đơn giản hóa toàn bộ quá trình mã hóa và giải mã trong Linear Block Code, khi phép cộng tương đương với XOR và phép nhân tương đương với AND. Nhờ đó, độ phức tạp tính toán được giảm đáng kể, tốc độ xử

lý được cải thiện, đặc biệt trong quá trình tính syndrome, khi chỉ cần thực hiện phép XOR trên các bit tương ứng để phát hiện lỗi.

2. Mã LDPC

LDPC (Low-Density Parity-Check) là một dạng Linear Block Code với ma trận kiểm tra chẵn lẻ H có tính chất thưa, tức là chỉ chứa một số lượng rất nhỏ các phần tử khác 0. Các phép toán trong mã LDPC được thực hiện trên trường nhị phân $GF(2)$.



Hình 2: Ma trận kiểm tra chẵn lẻ H

Tương tự như các mã khối tuyến tính khác, mỗi từ mã hợp lệ của LDPC phải thỏa mãn:

$$c * H^T = 0$$

Tại phía thu, để kiểm tra lỗi, syndrome được tính như sau:

$$s = r * H^T$$

Trong đó:

- + s : syndrome (vector kiểm tra lỗi)
- + r : vector nhận được sau khi truyền qua kênh
- + H : ma trận kiểm tra chẵn lẻ

Kiểm tra:

- + Nếu $s = 0$: không phát hiện lỗi
- + Nếu $s \neq 0$: có lỗi xảy ra

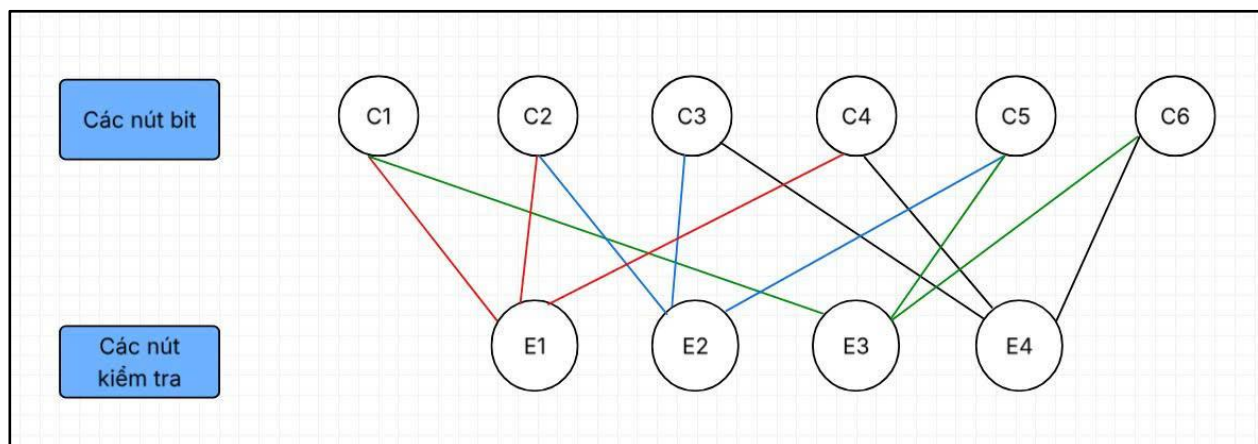
Nhờ cấu trúc thưa của ma trận H , số lượng phép tính được giảm đáng kể, giúp các thuật toán giải mã có thể thực hiện với độ phức tạp thấp.

3. Giải Đồ Tanner

Để biểu diễn trực quan ma trận kiểm tra chẵn lẻ H , mã LDPC thường được mô tả bằng đồ thị Tanner, là một đồ thị hai phía gồm hai loại nút:

- + Nút biến: đại diện cho các bit của từ mã (tương ứng với các cột của H)
- + Nút kiểm tra: đại diện cho các phương trình kiểm tra chẵn lẻ (tương ứng với các hàng của H)

Một cạnh được nối giữa nút biến i và nút kiểm tra j nếu phần tử $H(j,i) = 1$



Hình 3: Giải đồ Tanner

4. Thuật Toán Bit-Flipping

Thuật toán Bit-Flipping là một phương pháp giải mã đơn giản cho mã LDPC, dựa trên quyết định cứng (hard-decision). Thuật toán được thực hiện trên đồ thị Tanner, trong đó các nút kiểm tra đánh giá các điều kiện chẵn lẻ và gửi thông tin đến các nút biến.

Cụ thể:

- + Nếu một kiểm tra không thỏa mãn, nó sẽ “bỏ phiếu” cho các bit mà nó kết nối
- + Mỗi bit nhận các phiếu từ các kiểm tra liên quan
- + Bit có số lượng phiếu lỗi lớn nhất sẽ được lật (0 thành 1 hoặc ngược lại)

Quá trình này được lặp lại cho đến khi:

- + Tất cả các kiểm tra đều thỏa mãn (không còn lỗi), hoặc
- + Đạt số lần lặp tối đa

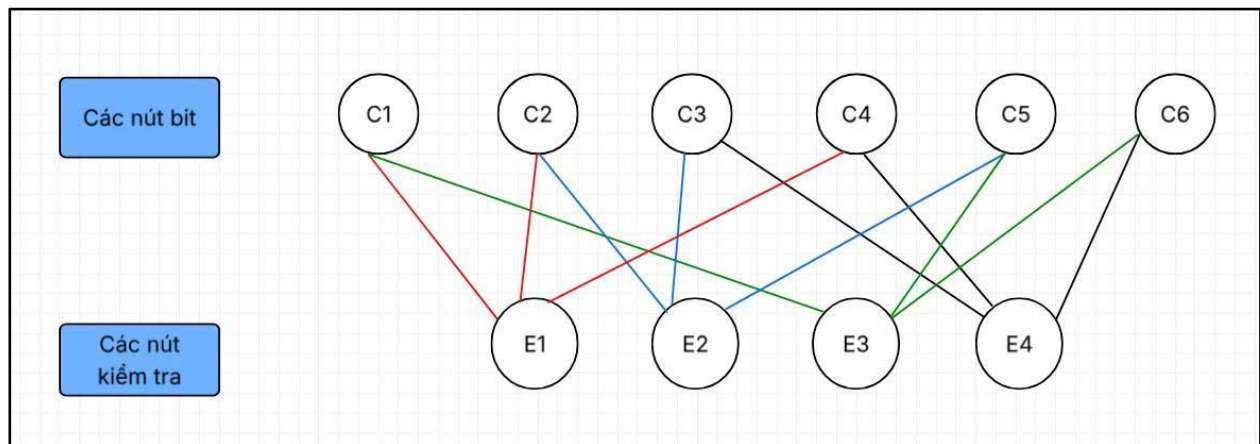
Thuật toán Bit-Flipping có ưu điểm là đơn giản, dễ triển khai và có độ phức tạp thấp. Tuy nhiên, hiệu năng của nó không cao bởi vì nó không đảm bảo luôn hội tụ về kết quả đúng trong mọi trường hợp.

II. THỰC HÀNH

Cho ma trận kiểm tra Parity H như sau:

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Câu 1: Vẽ giản đồ Tanner của mã hóa trên?



Câu 2: Cho chuỗi bit ở phía phát là: $c=[0\ 0\ 1\ 0\ 1\ 1]$, giả sử bit đầu tiên bị sai. Tham khảo tài liệu đính kèm, sử dụng thuật toán lật bit để tìm lại $\hat{c}$.

giả sử bit đầu tiên bị sai $\Rightarrow$ từ mã c được truyền qua kênh BSC và tín hiệu nhận được là:

$$y = [101011]$$

Khởi tạo $M = [101011]$

$$E_{1,1} = M_2 \oplus M_4 = 0 \oplus 0 = 0$$

$$E_{1,2} = M_1 \oplus M_4 = 1 \oplus 0 = 1$$

$$E_{1,4} = M_1 \oplus M_2 = 1 \oplus 0 = 1$$

$$E_{2,2} = M_3 \oplus M_5 = 1 \oplus 1 = 0$$

$$E_{2,3} = M_2 \oplus M_5 = 0 \oplus 1 = 1$$

$$E_{2,5} = M_2 \oplus M_3 = 0 \oplus 1 = 1$$

$$E_{3,1} = M_5 \oplus M_6 = 1 \oplus 1 = 0$$

$$E_{3,5} = M_1 \oplus M_6 = 1 \oplus 1 = 0$$

$$E_{3,6} = M_1 \oplus M_5 = 1 \oplus 1 = 0$$

$$E_{4,3} = M_4 \oplus M_6 = 0 \oplus 1 = 1$$

$$E_{4,4} = M_3 \oplus M_6 = 1 \oplus 1 = 0$$

$$E_{4,6} = M_3 \oplus M_4 = 1 \oplus 0 = 1$$

Bit 1: nhận 0 từ C1, nhận 0 từ C3

Mà hiện tại bit 1 nhận được: 1

=> (2 lỗi)

Bit 2: nhận 1 từ C1, nhận 0 từ C2

Mà hiện tại bit 2 nhận được: 0 (1 lỗi)

=> (1 lỗi)

Bit 3: nhận 1 từ C2, nhận 1 từ C3

Mà hiện tại bit 3 nhận được: 1

=> (0 lỗi)

Bit 4: nhận 1 từ C1, nhận 0 từ C4

Mà hiện tại bit 4 nhận được: 0

=> (1 lỗi)

Bit 5: nhận 1 từ C2, nhận 0 từ C3

Mà hiện tại bit 5 nhận được: 0

=> (1 lỗi)

Bit 6: nhận 0 từ C3, nhận 1 từ C4

Mà hiện tại bit 6 nhận được: 1

=> (1 lỗi)

Vậy bit 1 nhiều lỗi nhất (2 lỗi), sau khi lật bit 1, thì $M = [001011]$

Kiểm tra lại:

$$L1 = M1 \oplus M2 \oplus M4 = 0$$

$$L2 = M2 \oplus M3 \oplus M5 = 0$$

$$L3 = M1 \oplus M5 \oplus M6 = 0$$

$$L4 = M3 \oplus M4 \oplus M6 = 0$$

Vậy $\hat{c} = [001011]$

Câu 3: Bổ sung chương trình tại điều kiện vòng lặp, tính ma trận kiểm tra lỗi, và sửa lỗi để hoàn tất chương trình.

Hàm kiểm tra lỗi

```
function res = check_errors(H, current_frame)
syndrome = H * transpose(current_frame); % H * c^T
areErrors = any(mod(syndrome,2)); % nếu có ít nhất 1 bit khác 0 thì kết quả là 0
là sai
res = areErrors;
end
```

Hàm main

```
clear all;
```

```

close all;

% Ma trận kiểm tra
H=[1 1 0 1 0 0
   0 1 1 0 1 0
   1 0 0 0 1 1
   0 0 1 1 0 1];

% Từ mã truyền
c=[0 0 1 0 1 1];

% Từ mã nhận
r=[1 0 1 0 1 1];
y=r;

% Số lần lặp
maxiter=20;
iter=0;

% Đánh dấu giải mã chưa thành công
success = 0;

while (iter < maxiter && success == 0) % Điều kiện vòng lặp

    % Tính syndrome trước
    syndrome = mod(H * y', 2); % hàng đang sai

    % Khởi tạo ma trận lỗi
    E=zeros(4,6);

    for j = 1:4
        for i = 1:6
            if (H(j,i)==1) % nghĩa là H là kiểm tra bit ở vị trí i,
                % syndrome là check ở hàng j đó, nếu hàng đó sai, thì các bit được kiểm tra
                ở hàng đó có bị sai
                % Nếu check node j bị lỗi → ảnh hưởng tới bit i
                if syndrome(j) == 1
                    E(j,i) = 1;
                end
            end
        end
    end
end
end

```

```

end

E = E()
% Tìm vị trí có nhiều lỗi nhất
for i=1:6
    M(i)=sum(E(:,i));
end

[M,index] = max(M); % xuất ra giá trị và vị trí bị nhiều lỗi nhất

% Sửa lỗi (flip bit có nhiều phiếu lỗi nhất)
if M~=0
    %nếu ban đầu bit 0 hoặc chẵn, ví dụ 0+1 là 1 thì chia 2 dư 1, thì 0->1
    %nếu ban đầu bit 1 hoặc lẻ, ví dụ 1+1 là 2 thì chia 2 dư 0, thì 1-> 0
    y(index) = mod(y(index) + 1, 2);
end

% Kiểm tra sau khi đảo bit
areErrorsPresent = check_errors(H, y);

if areErrorsPresent == 0 % Không lỗi
    success = 1;
    disp("No error");
    break;
else % Có lỗi
    disp("Still errors");
end

iter=iter+1;
end

disp("Decoded result:");
disp(y);

```

Kết quả:

```

c =

    0    0    1    0    1    1

r =

```

1 0 1 0 1 1

E =

1 1 0 1 0 0
0 0 0 0 0 0
1 0 0 0 1 1
0 0 0 0 0 0

M =

2

index =

1

No error

Decoded result:

0 0 1 0 1 1

Giải thích code:

a. Khởi động vòng lặp

```
while (iter < maxiter && success == 0)
```

Chương trình sẽ tiếp tục điều tra nếu: Vẫn còn lỗi (success == 0) và chưa điều tra quá 20 lần (iter < maxiter). Nếu quá 20 lần mà không ra, chương trình sẽ tự bỏ cuộc để tránh bị kẹt vô tận (lặp vô hạn).

b. Tính syndrome (kiểm tra lỗi)

```
syndrome = mod(H * y', 2);
```

Nhân ma trận kiểm tra H với từ mã hiện tại y. Kết quả syndrome là một mảng 4 con số (ví dụ [1; 0; 1; 0]).

- + Số 1 nghĩa là bài test đó thất bại (báo lỗi).
- + Số 0 nghĩa là bài test đó pass (bình thường).

c. Xây dựng ma trận lỗi E

```
for j = 1:4
    for i = 1:6
        if (H(j,i) == 1)
            if syndrome(j) == 1
                E(j,i) = 1;
            end
        end
    end
end
end
```

Ma trận E dùng để lưu các “phiếu lỗi” từ các kiểm tra.

- + Nếu $H(j,i) = 1$ nghĩa là bit i tham gia vào kiểm tra j
- + Nếu kiểm tra j bị lỗi ($\text{syndrome}(j) = 1$) thì bit i nhận 1 phiếu lỗi

Nói cách khác: mỗi kiểm tra sai sẽ “bỏ phiếu” cho tất cả các bit liên quan.

d. Tổng hợp số phiếu lỗi cho từng bit

```
for i = 1:6
    M(i) = sum(E(:,i));
end
```

```
[M, index] = max(M);
```

- + Cộng dọc các cột của số E lại.
- + $M(i)$ là tổng số phiếu lỗi mà bit i nhận được
- + Bit nào nằm trong nhiều kiểm tra sai sẽ có giá trị $M(i)$ lớn

Sau đó tìm:

- + Giá trị lớn nhất
- + Vị trí bit có nhiều phiếu lỗi nhất (index)

e. Sửa lỗi (lật bit)

```
if M ~= 0
    y(index) = mod(y(index) + 1, 2);
end
```

Nếu tồn tại lỗi, thuật toán sẽ:

- + Chọn bit có số phiếu lỗi lớn nhất
- + Lật bit đó (0 thành 1, hoặc 1 thành 0)

Hàm $\text{mod}(\dots, 2)$ đảm bảo giá trị luôn là 0 hoặc 1.

f. Kiểm tra lại sau khi sửa

```
areErrorsPresent = check_errors(H, y);
```

Sau khi lật bit, vector y mới sẽ được kiểm tra lại:

- + Nếu không còn lỗi $\rightarrow$ dừng thuật toán
- + Nếu vẫn còn lỗi $\rightarrow$ tiếp tục lặp

Nếu đạt tới số lần lặp tối đa mà vẫn còn lỗi, thuật toán sẽ dừng và trả về kết quả hiện tại.